



INSTITUTO TECNOLÓGICO SUPERIOR DE MISANTLA

INGENIERIA EN SISTEMAS COMPUTACIONALES

MATERIA:

Taller de Base de datos.

DOCENTE:

Dr. Irahan Otoniel José Guzmán

NOMBRE DE LA ACTIVIDAD:

Reporte de ejercicios en google colab.

P R E S E N T A:

Nadia Guadalupe Salazar García

Misantla

ÍNDICE

INTRODUCCIÓN.....	3
EJERCICIO 1: DESAYUNOS (RESTAURANTE).....	4
CONEXIÓN A LA BASE DE DATOS.....	4
CREACIÓN DE LA TABLA.....	4
INSERCIÓN DE REGISTROS	5
MODIFICACIÓN DE REGISTROS	5
ELIMINACIÓN DE REGISTROS.....	6
EJERCICIO 2: ESTUDIANTES (ESCUELA).....	7
CONEXIÓN A LA BASE DE DATOS.....	7
CREACIÓN DE LA TABLA.....	7
INSERCIÓN DE REGISTROS	8
MODIFICACIÓN DE REGISTROS	8
ELIMINACIÓN DE REGISTROS.....	9
EJERCICIO 3: TIENDITA ESCOLAR (PRODUCTOS)	10
CONEXIÓN A LA BASE DE DATOS.....	10
CREACIÓN DE LA TABLA.....	10
INSERCIÓN DE REGISTROS	11
MODIFICACIÓN DE REGISTROS	11
ELIMINACIÓN DE REGISTROS.....	11
CONCLUSION.....	13

INTRODUCCIÓN

En este reporte puse en práctica las operaciones fundamentales para manejar una base de datos: crear tablas, agregar información, modificarla y eliminar registros. Usé el lenguaje SQL junto con la librería sqlite3 de Python en Google Colab, guardando todo en memoria para practicar sin riesgo de dañar datos reales.

Realicé tres ejercicios con temas distintos: un menú de desayunos, el control de calificaciones de estudiantes y el inventario de una tiendita escolar. En cada uno seguí la misma secuencia: establecer la conexión, definir la estructura de la tabla, insertar los datos, actualizar información bajo una condición específica y, por último, eliminar los registros que ya no se necesitaban. El objetivo fue comprobar que aunque los temas cambien, los comandos y la lógica para administrar la información son siempre los mismos.

EJERCICIO 1: DESAYUNOS (RESTAURANTE)

CONEXIÓN A LA BASE DE DATOS

Primero importamos las librerías que necesitamos y abrimos la conexión a la base de datos temporal. También definimos dos funciones: una para consultar la información y verla en forma de tabla, y otra para ejecutar los cambios como agregar, modificar o borrar datos.

```
[1]
✓ 0 s
import sqlite3
import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)
conn = sqlite3.connect(":memory:")
cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):
    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""
    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):
    """Ejecuta INSERT/UPDATE/DELETE/DDL y hace commit."""
    cur.execute(sql, params)
    conn.commit()
    print("OK ->", sql.strip().splitlines()[0], "...")
```

Figura 1 Conexión a la base de datos y definición de funciones auxiliares.

CREACIÓN DE LA TABLA

Se creó la tabla llamada desayunos con todos sus campos: identificador único, nombre del platillo, su categoría, precio y si está disponible. Se estableció que el nombre y el precio no pueden quedar vacíos, y que, si no se dice lo contrario, todo se marca como disponible.

```
cur.executescript('''
DROP TABLE IF EXISTS desayunos;

CREATE TABLE desayunos (
    id_platillo INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre VARCHAR(50) NOT NULL,
    tipo VARCHAR(30),
    precio DECIMAL(6,2) NOT NULL,
    disponible BOOLEAN DEFAULT TRUE
);
''')
conn.commit()
print("Tabla creada: desayunos")
```

... Tabla creada: desayunos

Figura 2 Creación de la tabla desayunos con sus campos y restricciones.

INSERCIÓN DE REGISTROS

Se agregaron en total siete platillos distintos con sus datos correspondientes. Uno de ellos se marcó como no disponible a propósito, para después poder probar cómo funciona la eliminación de registros.

```
[7]
✓ 0 s
run("""INSERT INTO desayunos (nombre, tipo, precio, disponible) VALUES
('Huevos a la mexicana', 'platillo', 72.50, TRUE),
('Chilaquiles verdes', 'platillo', 68.00, TRUE),
('Pan dulce con café', 'ligero', 35.00, TRUE),
('Enfrijoladas', 'platillo', 58.00, FALSE),
('Fruta con yogurt', 'ligero', 42.00, TRUE),
('Jugo verde', 'bebida', 22.00, TRUE),
('Champurrado', 'bebida', 28.00, TRUE);""")
query("SELECT * FROM desayunos")
```

Figura 3 Inserción de los platillos en la tabla de desayunos.

... OK -> INSERT INTO desayunos (nombre, tipo, precio, disponible) VALUES ...

	id_platillo	nombre	tipo	precio	disponible
0	1	Huevos a la mexicana	platillo	72.5	1
1	2	Chilaquiles verdes	platillo	68.0	1
2	3	Pan dulce con café	ligero	35.0	1
3	4	Enfrijoladas	platillo	58.0	0
4	5	Fruta con yogurt	ligero	42.0	1
5	6	Jugo verde	bebida	22.0	1
6	7	Champurrado	bebida	28.0	1

Figura 4 Verificación de los platillos insertados.

MODIFICACIÓN DE REGISTROS

Se aplicó un aumento del 8% al precio de todos los platillos completos. Esto se hizo seleccionando solamente los que pertenecen al tipo 'platillo', así se actualizaron todos juntos sin tener que cambiar cada uno por separado.

```

▶ # UPDATE: aumento de precio a los platillos completos
run("""UPDATE desayunos
SET precio = precio * 1.08
WHERE tipo = 'platillo';""")

query("SELECT id_platillo, nombre, tipo, precio FROM desayunos WHERE tipo = 'platillo'")

```

... OK -> UPDATE desayunos ...

	id_platillo	nombre	tipo	precio
0	1	Huevos a la mexicana	platillo	78.30
1	2	Chilaquiles verdes	platillo	73.44
2	4	Enfrijoladas	platillo	62.64

Figura 5 Actualización del precio de platillo.

ELIMINACIÓN DE REGISTROS

Se eliminó únicamente el platillo que estaba marcado como no disponible. Gracias a la condición que pusimos, solo se quitó ese registro y se mantuvieron todos los demás que sí siguen en el menú.

```

▶ # DELETE: se quita el platillo que no está disponible
run("""DELETE FROM desayunos
WHERE disponible = FALSE;""")

query("SELECT * FROM desayunos")

```

... OK -> DELETE FROM desayunos ...

	id_platillo	nombre	tipo	precio	disponible
0	1	Huevos a la mexicana	platillo	78.30	1
1	2	Chilaquiles verdes	platillo	73.44	1
2	3	Pan dulce con café	ligero	35.00	1
3	5	Fruta con yogurt	ligero	42.00	1
4	6	Jugo verde	bebida	22.00	1
5	7	Champurrado	bebida	28.00	1

Figura 6 . Eliminación del platillo marcado como no disponible.

EJERCICIO 2: ESTUDIANTES (ESCUELA)

CONEXIÓN A LA BASE DE DATOS

Se usó la misma conexión que ya teníamos, ya que trabajamos en la misma base de datos. Solo se agregó una tabla nueva para guardar la información de los alumnos.

```
[10]
✓ 0 s
import sqlite3
import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)
conn = sqlite3.connect(":memory:")
cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):
    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""
    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):
    """Ejecuta INSERT/UPDATE/DELETE/DDI y hace commit."""
    cur.execute(sql, params)
    conn.commit()
    print("OK ->", sql.strip().splitlines()[0], "...")
```

Figura 7 Conexión a la base de datos para el ejercicio de estudiantes.

CREACIÓN DE LA TABLA

Se creó la tabla llamada estudiantes con los campos: identificador, nombre, materia que cursa, semestre y calificación. Se definió que el nombre es obligatorio, y que, si no se indica la calificación, queda en 0.0 por defecto.

```
[11]
✓ 0 s
cur.executescript('''
DROP TABLE IF EXISTS estudiantes;

CREATE TABLE estudiantes (
    id_est INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre VARCHAR(50) NOT NULL,
    materia VARCHAR(30),
    semestre VARCHAR(10),
    calificacion DECIMAL(4,2) DEFAULT 0.0
);
''')
conn.commit()
print("Tabla creada: estudiantes")

... Tabla creada: estudiantes
```

Figura 8 Creación de la tabla estudiantes con sus campos y restricciones.

INSERCIÓN DE REGISTROS

Se registraron cuatro alumnos con sus datos: cada uno con su nombre, materia distinta, el semestre que cursa y su calificación. Así tenemos información variada para poder hacer cambios después.

```
[12] ✓ 0 s run("""INSERT INTO estudiantes (nombre, materia, semestre, calificacion) VALUES
('Luis Ramírez', 'Matemáticas', '2do', 88),
('María López', 'Español', '1ro', 92),
('Javier Cruz', 'Historia', '3ro', 79),
('Karla Méndez', 'Inglés', '2do', 95);""")

query("SELECT * FROM estudiantes")
```

... OK -> INSERT INTO estudiantes (nombre, materia, semestre, calificacion) VALUES ...

	id_est	nombre	materia	semestre	calificacion
0	1	Luis Ramírez	Matemáticas	2do	88
1	2	María López	Español	1ro	92
2	3	Javier Cruz	Historia	3ro	79
3	4	Karla Méndez	Inglés	2do	95

Figura 9 Lista de alumnos registrados.

MODIFICACIÓN DE REGISTROS

Se aumentaron 3 puntos a la calificación de todos los alumnos que van en 2do semestre, por haber entregado un trabajo extra. Gracias a la condición, solo se modificaron los de ese semestre y los demás quedaron igual.

```
[13] ✓ 0 s run("""UPDATE estudiantes
SET calificacion = calificacion + 3
WHERE semestre = '2do';""")

query("SELECT id_est, nombre, semestre, calificacion FROM estudiantes WHERE semestre = '2do'")
```

... OK -> UPDATE estudiantes ...

	id_est	nombre	semestre	calificacion
0	1	Luis Ramírez	2do	91
1	4	Karla Méndez	2do	98

Figura 10 Calificaciones actualizadas para el 2do semestre.

ELIMINACIÓN DE REGISTROS

Se eliminó al alumno que cursaba la materia de Historia, ya que dejó de asistir. Con la condición que pusimos, solo se borró ese registro y se mantuvieron los demás alumnos en la tabla.

```
15] Os  # DELETE: se elimina el alumno de la materia de Historia
run("DELETE FROM estudiantes WHERE materia = 'Historia';")

query("SELECT * FROM estudiantes")
```

✓ ... OK -> DELETE FROM estudiantes WHERE materia = 'Historia'; ...

	id_est	nombre	materia	semestre	calificacion
0	1	Luis Ramírez	Matemáticas	2do	91
1	2	María López	Español	1ro	92
2	4	Karla Méndez	Inglés	2do	98

Figura 11 Lista final después de eliminar el registro.

EJERCICIO 3: TIENDITA ESCOLAR (PRODUCTOS)

CONEXIÓN A LA BASE DE DATOS

Se usó la misma conexión de los ejercicios anteriores, ya que trabajamos en la misma base de datos. Solo se agregó una tabla nueva para llevar el control de los productos de la tiendita escolar.

```
[16]
✓ 0 s
import sqlite3
import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)
conn = sqlite3.connect(":memory:")
cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):
    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""
    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):
    """Ejecuta INSERT/UPDATE/DELETE/DDL y hace commit."""
    cur.execute(sql, params)
    conn.commit()
    print("OK ->", sql.strip().splitlines()[0], "...")
```

Figura 12 Conexión a la base de datos para el ejercicio de la tiendita escolar.

CREACIÓN DE LA TABLA

Se creó la tabla llamada productos con los campos: identificador del producto, nombre, tipo de producto, precio y cantidad disponible. El nombre y el precio son obligatorios; si no se indica cuántos hay, la existencia queda en 0 por defecto.

```
[17]
✓ 0 s
cur.executescript('''
DROP TABLE IF EXISTS productos;

CREATE TABLE productos (
    id_producto INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre VARCHAR(50) NOT NULL,
    tipo VARCHAR(30),
    precio DECIMAL(6,2) NOT NULL,
    existencia INTEGER DEFAULT 0
);
''')
conn.commit()
print("Tabla creada: productos")

... Tabla creada: productos
```

Figura 13 Creación de la tabla productos de la tiendita escolar.

INSERCIÓN DE REGISTROS

Se registraron cinco productos que se venden en la tiendita: cada uno con su nombre, tipo, precio y cantidad en existencia.

```
[18]
✓ 0 s
run("""INSERT INTO productos (nombre, tipo, precio, existencia) VALUES
('Papas fritas', 'snack', 15.00, 50),
('Refresco', 'bebida', 18.00, 40),
('Gomitas', 'dulce', 12.00, 30),
('Sándwich', 'comida', 25.00, 15),
('Agua embotellada', 'bebida', 10.00, 80);""")

query("SELECT * FROM productos")

... OK -> INSERT INTO productos (nombre, tipo, precio, existencia) VALUES ...
```

	id_producto	nombre	tipo	precio	existencia
0	1	Papas fritas	snack	15	50
1	2	Refresco	bebida	18	40
2	3	Gomitas	dulce	12	30
3	4	Sándwich	comida	25	15
4	5	Agua embotellada	bebida	10	80

Figura 14 Productos registrados en la tiendita escolar.

MODIFICACIÓN DE REGISTROS

Se subió 10% el precio de todas las bebidas por aumento de costos. Solo se cambiaron los de tipo 'bebida', los demás productos quedan igual.

```
run("UPDATE productos SET precio = precio * 1.10 WHERE tipo = 'bebida';")

query("SELECT id_producto, nombre, tipo, precio FROM productos WHERE tipo = 'bebida'")

... OK -> UPDATE productos SET precio = precio * 1.10 WHERE tipo = 'bebida'; ...
```

	id_producto	nombre	tipo	precio
0	2	Refresco	bebida	19.8
1	5	Agua embotellada	bebida	11.0

Figura 15 Precios de bebidas actualizadas.

ELIMINACIÓN DE REGISTROS

Se eliminaron las gomitas del sistema porque ya no se van a vender más. Solo se borró ese producto y todo lo demás sigue registrado.

s

```
run("DELETE FROM productos WHERE nombre = 'Gomitas';")
```

```
query("SELECT * FROM productos")
```

... OK -> DELETE FROM productos WHERE nombre = 'Gomitas'; ...

	id_producto	nombre	tipo	precio	existencia
0	1	Papas fritas	snack	15.0	50
1	2	Refresco	bebida	19.8	40
2	4	Sándwich	comida	25.0	15
3	5	Agua embotellada	bebida	11.0	80

Figura 16 Lista de productos final de la tiendita escolar.

CONCLUSION

Al terminar los tres ejercicios comprendí que las cuatro acciones básicas —crear, insertar, actualizar y borrar— son la base de cualquier sistema que maneje información. Aunque trabajé con temas diferentes: comida, escuela y productos, la estructura de los códigos fue muy parecida, lo que me ayuda a entender que estos mismos pasos se pueden aplicar a cualquier situación.

También me di cuenta de algo muy importante: al usar UPDATE y DELETE hay que escribir muy bien la condición, porque si me equivoco puedo cambiar o borrar más información de la que quiero. Practicar con ejemplos sencillos me permitió ver claramente qué hace cada instrucción y cómo afecta a los datos. En resumen, aprendí no solo a escribir los comandos, sino a cuidar la información y a pensar bien antes de ejecutar un cambio.